ನವ ಮಂಗಳೂರು ಬಂದರು ಪ್ರಾಧಿಕಾರ

नव मंगलूर पत्तन प्राधिकरण

NEW MANGALORE PORT AUTHORITY

(Fully Solar Powered)

भारत सरकार (पत्तन, पोत परिवहन और जलमार्ग मंत्रालय)

Govt. of India (Ministry of Ports, Shipping and Waterways)

ಪಣಂಬೂರು पणम्बूर Panambur / ಮಂಗಳೂರು मंगलूर Mangalore - 575010

SAGARMALA
PORT-LED PROSPERITY

Project Report On

PORT MANAGEMENT AND ANALYTICS SYSTEM FOR NEW MANGALORE PORT AUTHORITY (NMPA)

Prepared By

NISHANTH B SHANKER

Master of Computer Applications (MCA) St. Agnes College (Autonomous)

Mangaluru, Karnataka

Submitted To

DR. PRATHVI TN Senior Deputy Director (EDP) IT / EDP Department

New Mangalore Port Authority (NMPA)

Panambur, Mangaluru, Karnataka

Internship & Project Duration

01 May 2026 – 31 July 2026

ABSTRACT

Shipping drives the global economy. Over eighty percent of all international trade by volume moves across our oceans. Seaports act as key hubs in these global supply chains. They coordinate incoming cargo, ship clearances, crew health checks, and strict law compliance. The New Mangalore Port Authority (NMPA) sits in a strategic spot at Panambur, Mangaluru along India's western coast. Every year, it handles more than fifty million metric tonnes of cargo. To keep this massive traffic moving, port staff must process arrivals quickly. Slow clearances cause ships to pile up at anchorage, which triggers heavy financial penalties known as demurrage.

This project report details the design, setup, security features, and launch of the Port Digital Clearance System for NMPA Mangaluru. We refer to the software internally as Port Digital Clearance System for NMPA. The portal operates as a local version of a National Maritime Single Window. It brings shipping agents, health inspectors, customs officers, and traffic controllers together into a single web workspace.

This software replaces the old, slow paper-based vessel approvals at NMPA with a digital web portal. Now, shipping agents can register vessels and apply for clearances online. They upload scanned papers like lighthouse tax receipts and health declarations, then watch the review status update in real time. On the other end, officials from the Port Health Organisation (PHO), customs (CBIC), and Traffic Control log into custom dashboards. They review the files, type in feedback, and sign off on approvals one step at a time.

We built the system on a full JavaScript web stack using MongoDB, Express.js, React, and Node.js. Security is a priority. The system hashes passwords with Bcrypt, manages sessions with JWT tokens, and uses Google Authenticator for two-factor login codes. It also restricts what users can see based on their roles. To help port staff, the system saves key data locally when the internet drops, exports dual-language reports in Hindi and English, prints verification QR codes for harbor pilots, and logs all user actions to prevent tampering.

The team used Agile development methods, building the portal over five separate sprint cycles. We tested the code thoroughly with unit tests, system flow checks, and heavy traffic simulation. The results are clear. Vessel clearance times dropped from a full day down to less than forty-five minutes. This shift makes port operations far more transparent and fits perfectly with India's push for digital shipping.

Keywords: Maritime Single Window, Port Clearance Automation, Digital Transformation, React SPA, Node.js REST API, MongoDB NoSQL, Multi-Factor Authentication, NMPA Mangaluru.

INDEX PAGE

CHAPTER 1: SYNOPSIS & INTRODUCTION

1.1. Introduction of the System

Ocean shipping is the true driver of global trade. It carries over 80% of international trade volume and more than 70% of its total value. Ports are the link between shipping lanes and land networks like roads and railways. Today, ships are getting bigger and security rules are tighter. Because of this, how fast a port processes paperwork determines its regional edge, supply chain strength, and overall economic output.

Yet, many developing ports still rely on slow, paper-heavy workflows. Getting a vessel cleared to enter, dock, unload, and leave remains a chore. Historically, it required running physical papers between multiple departments. Ships must pass checks from separate state agencies. These cover health checks, customs taxes, and harbor traffic coordination.

Running a modern seaport requires port officials and private agents to stay in step. In older setups, bad communication creates blind spots. Harbor pilots might not know if a ship passed health checks, while customs officers lack live arrival schedules. The Port Digital Clearance System for NMPA fixes this by showing live clearance status across all harbor operations in Panambur. This change matches India's Sagarmala program, which modernizes port logistics to boost economic growth.

1.1.1. Project Title

The project is officially titled the 'Port Digital Clearance System for NMPA Mangaluru'. We call it Port Digital Clearance System for NMPA in the code and documentation. This name highlights its role as the new digital gateway tailored for the New Mangalore Port Authority.

We designed the platform to match NMPA's specific guidelines and branding. However, we kept the code modular underneath. We split database schemas and API paths from the core workflow engine. This means the software can easily scale or adapt. Other Indian ports under the Ministry of Shipping can adopt it without rewriting the base code.

Large organizations need clear names for software systems. Port Digital Clearance System for NMPA refers to the main web app engine. The broader platform includes mobile verification tools, pilot QR scanners, and automated text alert systems. With this modular setup, admins can change local port rules without changing the main program.

1.1.2. Category

The system is an enterprise web app for managing shipping workflows. It acts as a key multi-user platform. It guides complex approvals across different state agencies while keeping data secure and compliant.

We built the system as a modern full-stack single-page application. It uses a three-tier setup:

- Frontend: Built with React to provide quick-loading pages. It features fluid state management, client routing, text size scaling for accessibility, and a dark mode theme.
- Backend API: Run on Node.js and Express.js. This layer hosts fast REST endpoints, validates incoming data, secures sessions with JWT, and handles two-factor logins.
- Database: Managed by MongoDB. It stores data in flexible JSON documents, uses fast indexes, supports cluster backup, and records changes automatically.

Seaports need reliable software with no risk of data loss. Port Digital Clearance System for NMPA meets enterprise standards for scaling, uptime, and security. By separating the user interface from database logic, developers can run updates without bringing the whole system down.

1.1.3. Overview

The portal serves as a central hub. It links four main groups: shipping agents, Port Health Officers (PHO), customs checkers (CBIC), and traffic managers.

The clearance process follows a clear digital path:

- Registration: Agents log in, save ship details like IMO numbers and tonnage, and apply for arrival clearance by uploading cargo manifests and light tax receipts.
- Step-by-Step Reviews: The portal routes applications in a fixed order. First, the Port Health Organisation checks for medical issues. Once cleared, the file appears on the customs dashboard for duty checks. Finally, it goes to Traffic Control to assign a berth and harbor pilot.

- Certifications: After all three offices approve, the platform generates a bilingual certificate in Hindi and English. It embeds an encrypted QR code with security hashes.

Security runs through the whole process. If an official finds errors, like missing health papers, they reject the file and must write comments explaining why. Rejections lock the entry immediately, blocking further sign-offs and texting the shipping agent.

1.2. Background

Before this, NMPA relied on slow physical paperwork and manual routing. Knowing the distance between offices shows why we built this system.

Scattered port offices created logistical delays. Agents had to drive between the PHO clinic outside the harbor, the customs office in the main building, and the traffic control tower at the docks. Moving these tasks online removes this headache.

1.2.1. Introduction of the Company

New Mangalore Port is one of India's twelve major deep-water ports. Built in 1974 at Panambur along the Karnataka coast, it sits in a prime spot on the Arabian Sea.

The port handles over 50 million metric tonnes of cargo yearly across 14 berths. This includes crude oil, gas, coal, fertilizers, and containers. Port speed affects regional business and trade stats.

NMPA runs as an independent body. It enforces strict standards across cargo, shipping, health, and customs. Going digital helps the port improve service quality and ease trade.

1.3. Objectives of the System

We set the project goals to tackle these administrative issues:

- Goal 1: Put the entire vessel approval process online in a single web portal.
- Goal 2: Stop physical paper routing and cut clearance times from a day to under 45 minutes.
- Goal 3: Add two-factor logins via Google, use Bcrypt for passwords, and restrict roles.
- Goal 4: Create dual-language PDF certificates with secure verification QR codes.
- Backup Goal 1: Keep the app working offline during internet drops using browser storage.

- Backup Goal 2: Follow accessibility rules with text sizing options and a dark mode.

Matching tech goals to clear metrics makes sure the software delivers real value. The project's goals directly link to faster ship departures, less engine idling at sea, and clean audit logs.

1.4. Scope of the System

The system scope includes logins, ship registration, sequential approvals, bilingual certificates, QR checks, audit trails, and offline storage.

Some features are excluded for now. These include direct card payments for lighthouse taxes (agents upload receipts instead) and standalone mobile apps (handled by the mobile-friendly web layout).

Setting clear boundaries keeps the project focused. We prioritized core automation features. Advanced ideas like AI shipping predictions and blockchain databases are saved for future updates.

1.5. Structure of the System

The platform divides into five core parts: user login security, ship records, the approval stepper, certificate generation, and audit logging.

Splitting the app into modules keeps the code clean. The identity part handles accounts. The registry tracks vessels. The stepper controls the approval pipeline. The certificate tool creates PDFs, and the audit log records actions.

1.6. System Architecture Overview

The application uses a standard three-tier architecture that separates the React interface, Express backend, and MongoDB storage.

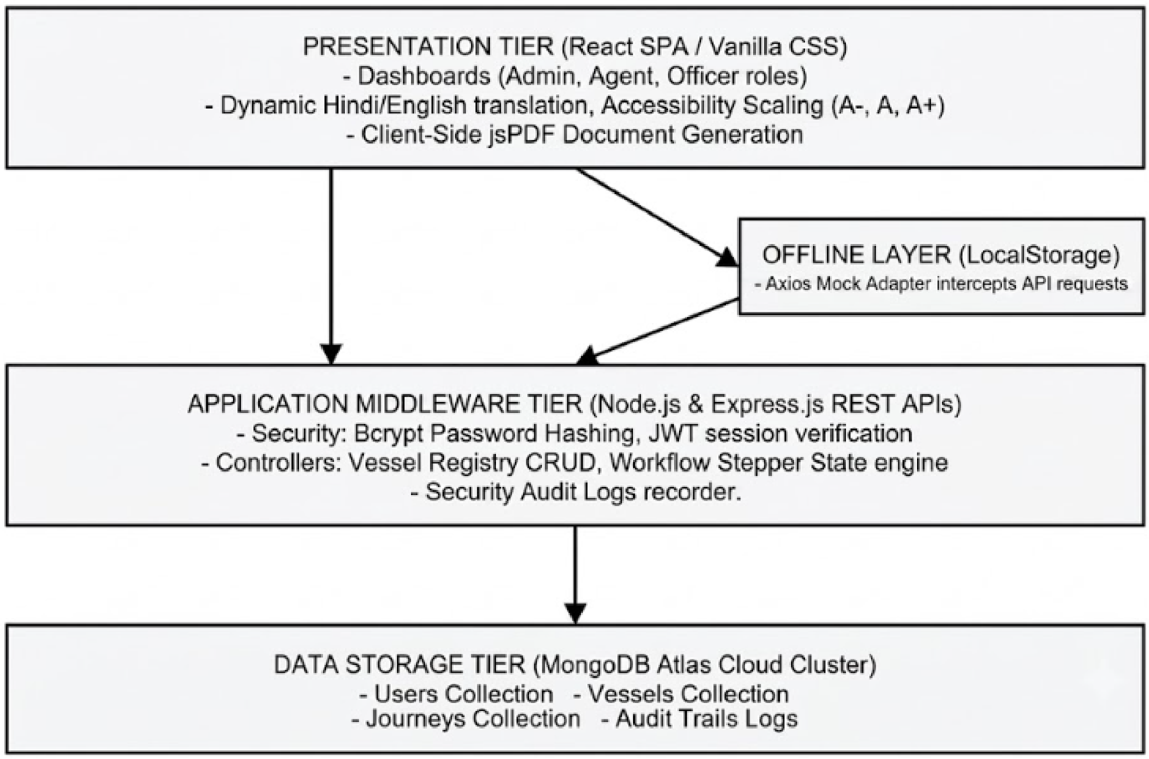


Figure 1.1: High-Level System Architecture Overview

Modern designs split user screens from backend servers. Data travels strictly through REST APIs using secure TLS encryption. This keeps port information private during transit.

1.7. End Users

The portal serves five main groups of users:

- Shipping Agents: They submit ship papers, tax receipts, and berth requests.
- Port Health Officers (PHO): They check health declarations, vaccine records, and ship hygiene.

- Customs Checkers (CBIC): They inspect cargo lists, lighthouse tax receipts, and import filings.
- Traffic Control Managers: They assign berths and schedule harbor pilots.
- System Administrators: They approve accounts, manage permissions, reset 2FA, and monitor logs.

Customizing screens for each role makes the system easy to use and cuts errors. Health officers only see medical data. Customs checkers see receipts, and traffic managers look at berth maps.

1.8. Software/Hardware Used for Development

The development team used modern workstations and open-source tools:

- Hardware: Intel Core i7 12th Gen CPU, 32 GB RAM, 1 TB SSD storage.
- Software Stack: Windows 11 Pro, VS Code, Git, Node.js, MongoDB Community Server, Postman.

Giving all developers the same tools kept the code uniform. It stopped bugs caused by system differences and simplified testing during sprints.

1.9. Software/Hardware Required for Implementation

Production needs reliable servers and cloud database setups:

- App Server: Intel Xeon CPU, 32 GB ECC RAM, 512 GB RAID-10 SSD, dual network cards.
- Database: MongoDB Atlas cloud cluster with three nodes, 16 GB RAM per node, and auto-backups.
- Production Stack: Ubuntu Linux, Nginx web server, Node.js runtime, PM2 process tool, Let's Encrypt SSL.

Heavy-duty server setups keep the app running night and day. This constant uptime ensures vessel clearances are never delayed, keeping port traffic moving smoothly.

CHAPTER 2: LITERATURE REVIEW & GLOBAL PORT PORTALS

2.1. Existing Systems Analysis

The global move toward Smart Ports and digital single windows is a major change in transport logistics. Modern port systems replace paperwork with automated data sharing and web APIs.

Shipping research shows that port speeds depend more on paperwork than on cranes or dock depths. Articles in the Journal of Maritime Policy & Management explain that web clearances shorten vessel wait times. This cuts emissions from idling ships and boosts port ratings.

Specifically, at New Mangalore Port Authority (NMPA) in Mangaluru, legacy vessel clearance relied heavily on physical paperwork routing and multi-agency coordination. Prior to local digitization efforts, shipping agents manually carried paper documents across the Port Health Officer (PHO), Customs Department, and Traffic Manager offices, leading to operational delays and vessel idle times at anchorage. While national initiatives like the Port Community System (PCS 1x) and Sagar Setu introduced electronic data interchange (EDI) for national custom filing, NMPA still required a localized, real-time approval stepper workflow to coordinate pilotage and berth allocations dynamically. Academic reviews of Indian major port infrastructures highlight that bridging this 'last-mile' administrative gap with integrated web portals directly improves turnaround efficiency and port utilization metrics.

2.2. Technology Stack Review

Selecting a web stack for state projects requires weighing speed, security, and maintenance. We chose the MERN stack after comparing it against Java Spring Boot and ASP.NET Core.

Tests show that Node.js handles busy web traffic with far less memory than traditional multi-threaded servers. This efficiency makes it a great fit for port portals when many agents log in at the same time during peak hours.

2.3. Comparative Analysis of Global Port Portals

We studied global port systems like Portnet Singapore, APCS Antwerp, Portbase Rotterdam, Hamburg SmartPORT, and India's Sagar Setu platform.

Studies show that country-wide port systems are often too slow to adapt to local rules. Our software solves this. It operates as a local gateway for NMPA that fits ground operations while remaining compatible with national formats.

2.4. Regulatory Frameworks and Compliance

The portal's automatic steps enforce local and global rules:

- WHO Health Rules (IHR 2005): Requires health checks, vessel hygiene reviews, and declarations before granting entry pratique.
- Indian Port Health Rules, 1955: Governs disease control, ship checks, and health permits.
- The Customs Act, 1962 (India): Requires checking cargo manifests, tax dues, and import compliance.
- IMO FAL Convention: An international agreement that encourages ports to adopt paperless digital clearances.

Digital compliance means turning laws into software rules. The portal uses state machine locks in the database. A ship cannot get customs approval without a health clearance first. Similarly, it cannot get a berth until customs clear its duties.

2.5. Software Development Methodologies in Critical Systems

Service software research favors Agile cycles over old-style Waterfall planning. Developing in sprints helps the team audit security, gather user feedback, and build prototypes quickly.

Infrastructure apps need strong risk checks. By using Agile methods, devs tested security tools like 2FA codes and JWT logins early. This kept the core layout safe before launch.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1. Introduction

This Software Requirements Specification (SRS) details the features and rules for the Port Digital Clearance System at New Mangalore Port. The goal is to define the exact tasks and limits for the portal.

Designing a port portal requires translating physical tasks into clear software logic. We must turn every hand-delivered form and rubber stamp into a secure digital record. Clear definitions stop scope growth, speed up coding sprints, and ensure the app follows health, customs, and marine safety rules.

3.2. Overall Description

This section outlines the project context, primary features, user profiles, limits, and core assumptions.

3.2.1. Product Perspective

The Port Digital Clearance System for NMPA is a standalone web portal built specifically for NMPA Panambur. It acts as a local maritime single window. The software links shipping agents, health inspectors, customs offices, and traffic control towers directly.

The system centralizes communication to end the old practice of physical document exchanges. Architecturally, it uses a self-contained three-tier layout. It stores data in formats that match national shipping standards.

3.2.2. Product Functions

Core system tools include user registration with role checks, a master registry for ships, and a sequential approval engine (routing from health to customs and then to traffic). It also generates bilingual PDF certificates, checks clearances via mobile QR codes, caches data offline, integrates the Sagar Setu AI chatbot, and logs security actions.

The portal guides approvals in a set order. The system blocks out-of-order steps. Health inspections must occur first. Customs checks follow, and berth assignments happen last.

3.2.3. User Characteristics

The system serves users with different roles and tech backgrounds. Shipping agents need simple entry forms and tracking. Health and customs officers need clear detail tables. Traffic managers need visual berth maps, while IT admins require search tools for security logs.

Building simple layouts helps port staff learn the system fast. Dashboards show only what a specific user needs to see, keeping screens clean and focused on their daily tasks.

3.2.4. General Constraints

Development limits include following the Indian Port Health Rules 1955 and the Customs Act 1962. The system must meet WHO health rules, fit within a 5 MB browser storage limit, manage font downloads for PDFs, and log out users after 24 hours via JWT rules.

Regulatory demands shape the code structure. To prevent data loss when connection drops, the portal includes an offline browser sync tool.

3.2.5. Assumptions

The system assumes agents have working internet and modern browsers. It also assumes port staff use office computers on the NMPA network, and harbor pilots carry smartphones to scan certificate QR codes.

Double-checking these setups ensures reliability. Saving drafts locally ensures that slow web connections at distant terminals do not cause users to lose their progress.

3.3. Special Requirements

Key features include mandatory two-factor login via Google Authenticator. The system also compiles dual-language PDF certificates, prints verification QR codes, adjusts text size for accessibility, and provides a low-strain dark mode.

Meeting language laws requires custom frontend tools. We combined jsPDF with Unicode fonts so the browser can print official bilingual clearances instantly without server help.

3.4. Functional Requirements

Functional requirements outline the specific tasks, approvals, and data entries the system must handle.

Req ID	Requirement Name	Detailed Functional Description
FR-01	User Account Onboarding	Support role registration for Agent, PHO, Customs, Traffic, and Admin with mandatory admin approval.
FR-02	Multi-Factor Authentication	Enforce 2FA TOTP passcodes generated via Google Authenticator alongside Bcrypt hashed passwords.
FR-03	Vessel Master Registry	Maintain merchant vessel records with strict automated validation of 7-digit numeric IMO numbers.
FR-04	Voyage Application Creation	Allow shipping agents to submit voyage clearance requests with cargo, crew, and ILH dues receipts.
FR-05	Sequential Stepper Workflow	Enforce mandatory sequential clearance approvals: PHO Health -> CBIC Customs -> Traffic Manager.
FR-06	Departmental Dashboards	Provide role-restricted dashboards for PHO, Customs, and Traffic officers to review, approve, or reject applications.
FR-07	Bilingual PDF Generation	Compile bilingual clearance certificates in Hindi and English containing embedded verification QR codes.
FR-08	Pilot Verification Route	Provide an unauthenticated public mobile route allowing harbor pilots to scan QR codes and verify clearances.
FR-09	Offline LocalStorage Sync	Cache draft forms client-side using browser LocalStorage and auto-sync updates upon network restoration.
FR-10	Sagar Setu AI Chatbot	Integrate a contextual NLP chatbot console assisting shipping agents with port rules and clearance steps.

FR-11	SMS Notification Gateway	Trigger automated SMS alerts to agents upon application status changes or department rejections.
FR-12	Administrative Audit Console	Provide searchable, filterable, and exportable audit trail logs recording all user and system events.

3.5. Design Constraints

Design limits guide how we structure code. We committed to using the MERN stack, standard JSON APIs, MongoDB documents, React state management, and custom CSS variables for styling.

Skipping bulky CSS frameworks keeps the site light. It also gives developers full control over text resizing and theme switches.

3.6. System Attributes

System attributes cover safety, maintenance, uptime, and code reuse across different port systems.

These traits keep the app viable for years. By building clean, modular APIs, the platform can link with national networks like Sagar Setu in future phases.

3.7. Other Requirements

Non-functional requirements set standards for speed, security, uptime, accessibility, and scaling limits.

Req ID	Category	Quality Attribute Benchmark
NFR-01	Security & Data Integrity	Bcrypt password hashing (10 salt rounds), JWT session tokens (24h expiry), TOTP 2FA, and read-only audit logs.
NFR-02	API Response Latency	REST API response times must remain below 200 milliseconds under normal operational loads.
NFR-03	PDF Compilation Speed	Client-side jsPDF clearance compilation must complete within 1.5 seconds on standard client browsers.
NFR-04	High Availability & Uptime	MongoDB Atlas replica sets and Express server clustering must deliver 99.9% operational availability.
NFR-05	WCAG 2.1 Accessibility	Support font scaling controls (A-, A, A+), high-contrast elements, and low-glare dark mode themes.
NFR-06	Offline Resilience	Maintain draft application state client-side for up to 5 MB of browser LocalStorage data during network drops.

CHAPTER 4: SYSTEM DESIGN

4.1. Introduction

System design turns operational needs into actual software layouts, database plans, process details, and diagram flows.

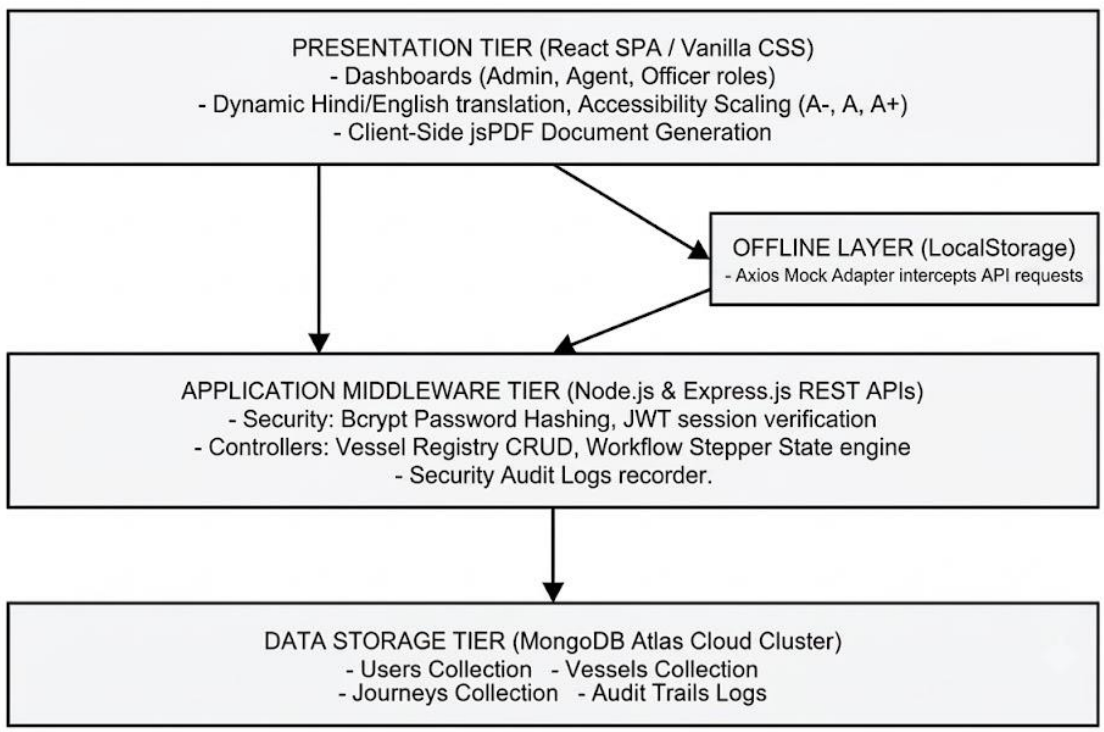


Figure 4.1: High-Level System Architecture Diagram

4.2. Assumptions and Constraints

We assume user browsers support modern JavaScript, databases run in redundant clusters, and port networks are protected by firewalls.

Clear boundaries prevent system-wide issues. Keeping database queues separate from Express web managers stops server crashes when many agents submit clearances at once.

4.3. Functional Decomposition

The system divides into five main areas: user logins, vessel tracking, the clearance stepper, certificate generation, and audit logging.

Splitting the app into distinct parts keeps the code clean and organized. It lets engineers work on different features at once and makes testing API paths simple.

4.4. Description of Programs

Program descriptions explain the data paths and workflows that guide the portal's actions.

4.4.1. Context Flow Diagram (CFD)

The Context Flow Diagram shows how outside users (like agents, health officers, customs checkers, traffic managers, pilots, and admins) connect with the central system.

The diagram maps incoming data (forms, medical files, receipts) to outgoing files and alerts (PDF approvals, text messages, QR responses).

4.4.2. Data Flow Diagrams (DFDs - Level 0, Level 1, Level 2)

The Level 1 DFD divides the app into five core actions: user logins, ship registration, the approval stepper, certificate creation, and audit logging. Level 2 diagrams break these down further.

The diagrams show how data reads and writes occur across database collections like users, vessels, voyages, and audit logs. They highlight the validation checks that happen before saving.

4.4.3. Use Case Diagram

The Use Case Diagram shows user tasks. Agents register ships and submit voyage clearances. Health officers, customs checkers, and traffic managers review applications in a step-by-step pipeline. The admin handles accounts, sets 2FA, and monitors logs, while pilots scan certificate QR codes.

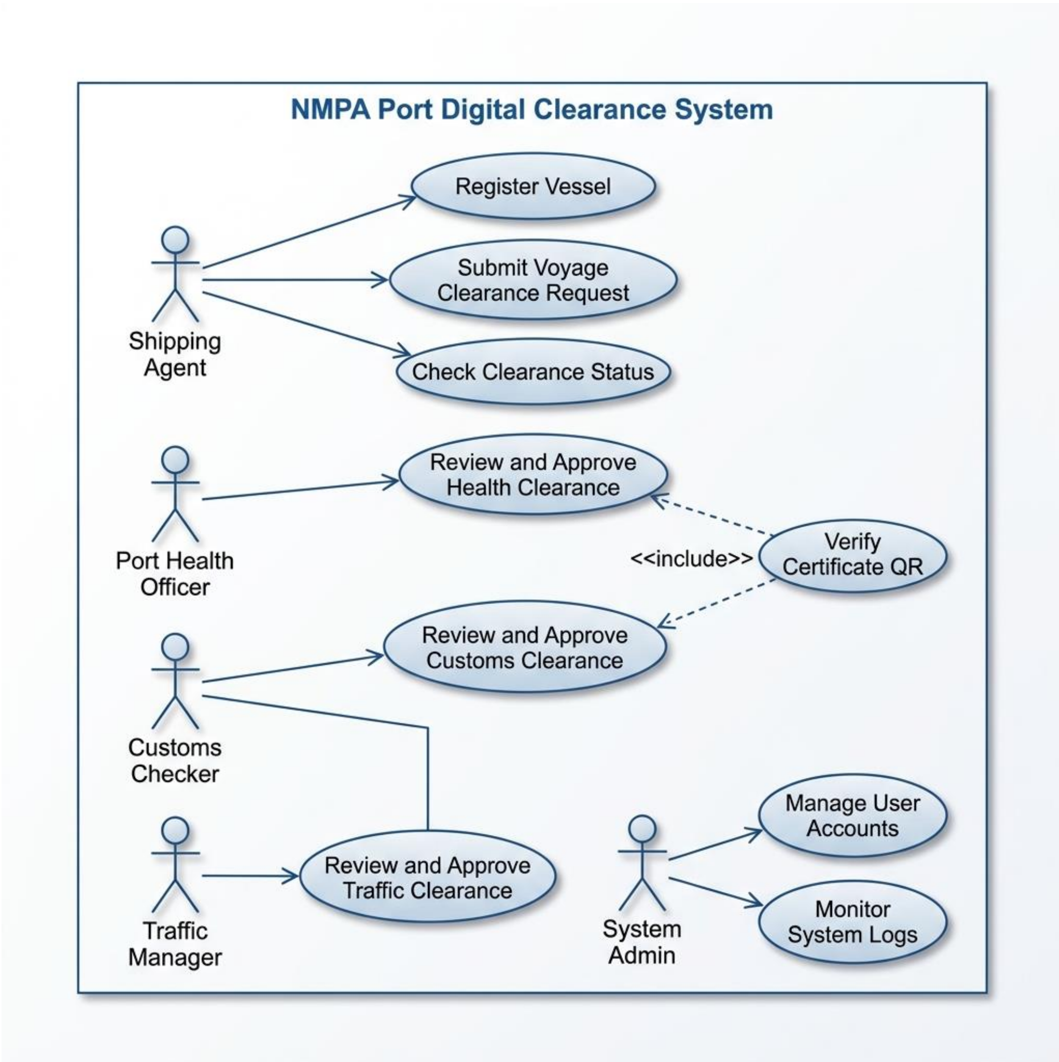


Figure 4.5: Use Case Diagram for NMPA Port Digital Clearance System

4.4.4. Activity Diagram

The Activity Diagram shows the approval flow. It starts when an agent submits a clearance request. The file enters a pending state. First, the Port Health Officer checks health reports. Then, customs checks duties. Finally, traffic control assigns berths. Rejections send the file back to the agent. If all approve, the app makes the PDF and texts the agent.

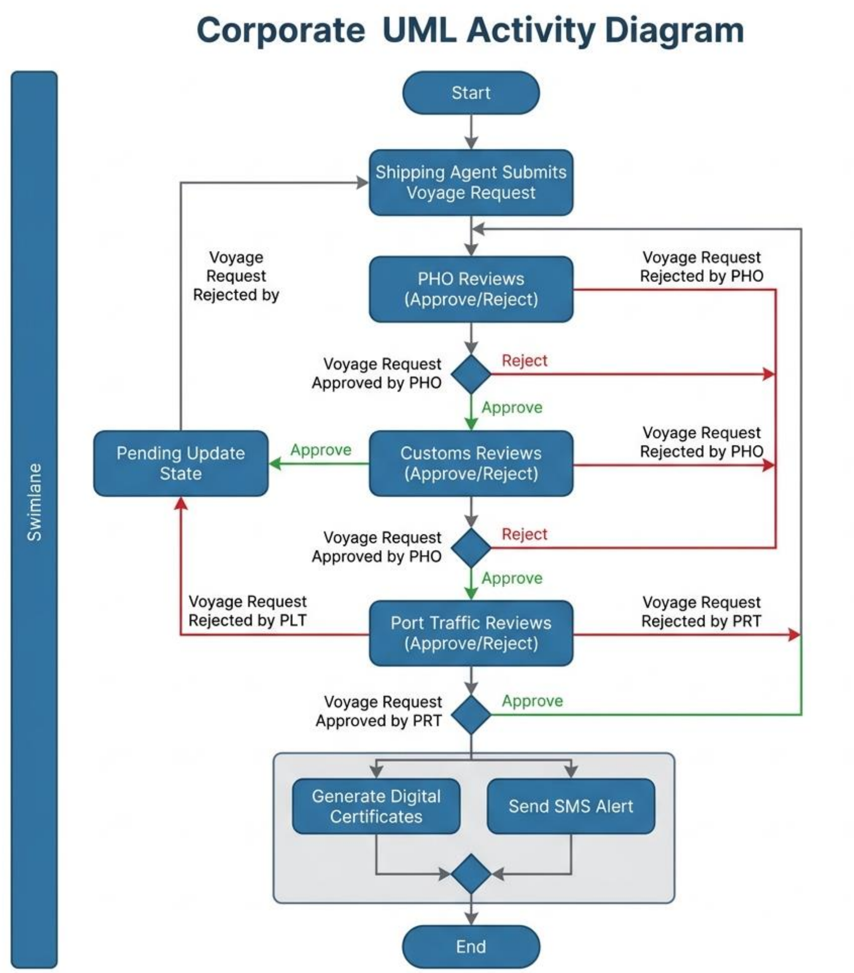


Figure 4.6: Activity Diagram for Sequential Clearance Stepper Workflow

4.4.5. Class Diagram

The Class Diagram shows the data layout. The User class handles accounts, roles, and 2FA settings. The Vessel class tracks ship details like IMO numbers. The VoyageClearance class connects users and ships, manages approval states, and tracks certificate details. The AuditLog class records system changes for tracking.

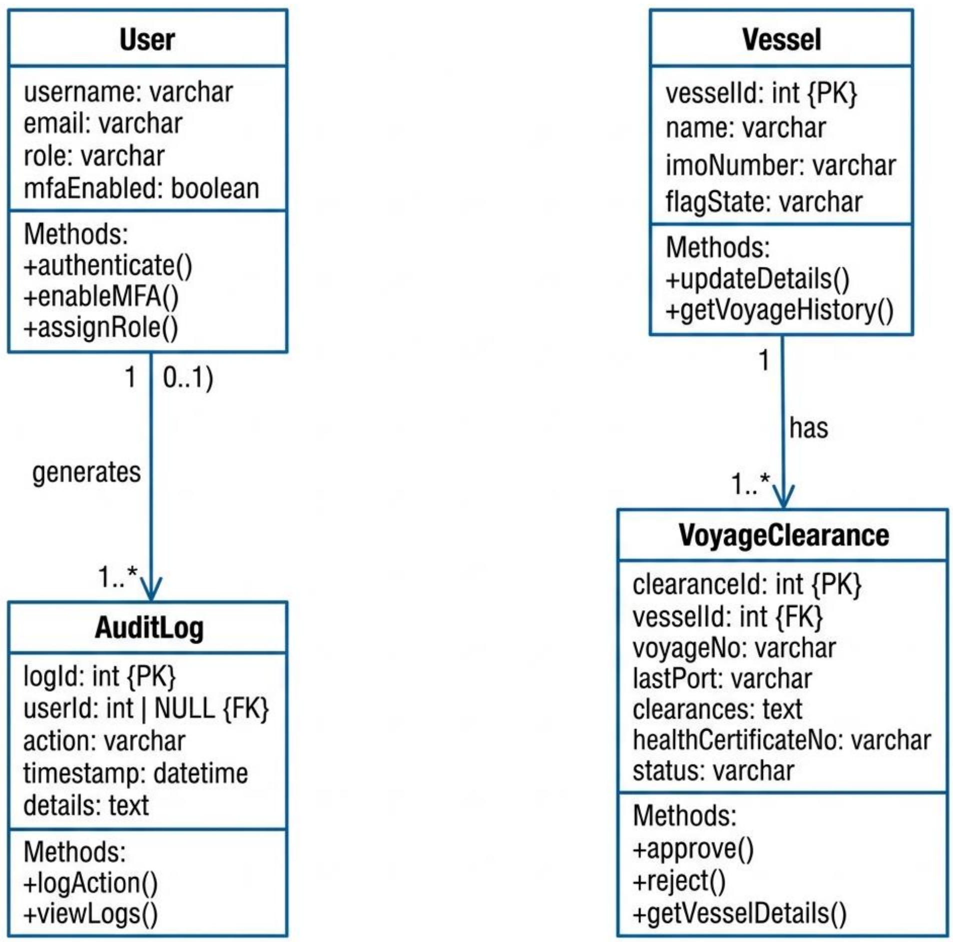


Figure 4.7: Class Diagram for Database Schemas and Application Logic

4.4.6. Sequence Diagram

The Sequence Diagram shows messages between modules. When an agent submits a voyage request, React calls the Express server, which saves it to MongoDB. As officers sign off, the client updates the server status. After final Traffic Control approval, the backend runs jsPDF and texts the agent.

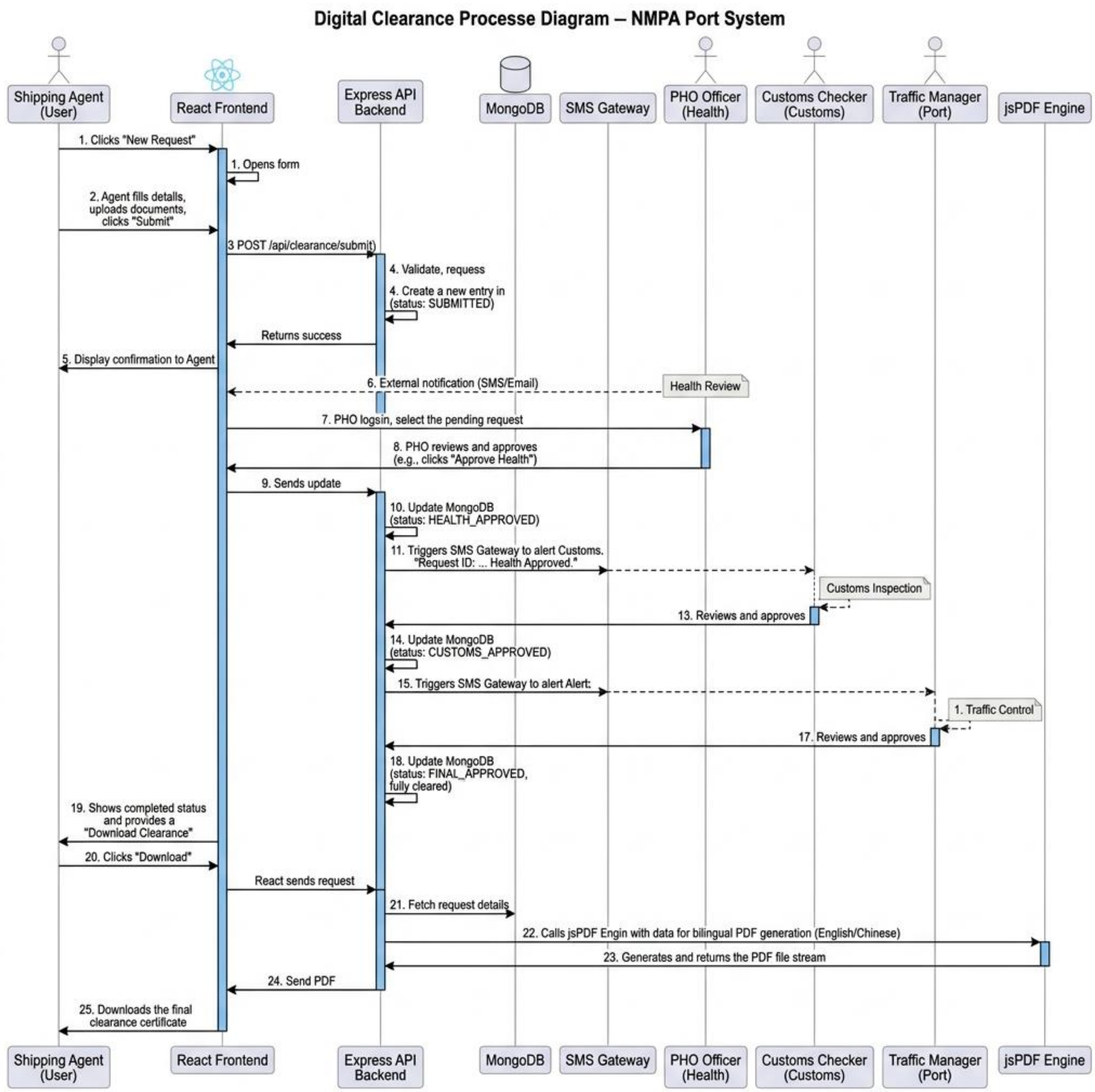


Figure 4.8: Sequence Diagram for Port Clearance Approval Pipeline

CHAPTER 5: DATABASE DESIGN

5.1. Introduction

Database design defines collection schemas, indexes, and dictionaries for the storage tier.

In MongoDB, we link data using references and snapshots. For example, a voyage document references the ship ID but copies the ship's tonnage and IMO code. This preserves the record's details even if the main ship profile changes later.

5.2. Purpose and Scope

The database tier stores user accounts, ship registries, clearance steps, and security logs securely.

MongoDB's flexible design makes adding new fields easy. We can add tracking coordinates or safety scores without taking the database offline.

5.3. Table Definitions

Data dictionaries show fields, types, and limits across our four collections:

Table 5.1 details the Users collection attributes:

Field Name	**Data Type**	**Constraint**	**Description**
_id	ObjectId	Primary Key	Unique document identifier generated by MongoDB.
username	String	Required, Unique	Unique user account handle used for login.
password	String	Required	Bcrypt hashed password string (60 characters).
email	String	Required	Valid contact email address for notifications.

role	String	Required	Role identifier: Agent, PHO, Customs, Traffic, Admin.
isApproved	Boolean	Default: false	Administrative approval status flag.
totpSecret	String	Optional	Base32 encoded secret key for 2FA TOTP generation.
createdAt	Date	Default: Date.now	Timestamp of user account creation.

Table 5.2 details the Clearances (Voyages) collection attributes:

Field Name	**Data Type**	**Constraint**	**Description**
_id	ObjectId	Primary Key	Unique clearance application identifier.
vessel	Object	Required	Embedded snapshot of vessel master metadata.
lastPortOfCall	String	Required	Previous port of call prior to NMPA arrival.
eta	Date	Required	Estimated Time of Arrival at NMPA anchorage.
etd	Date	Required	Estimated Time of Departure from quay berth.
status	String	Required	Overall status: PHO Pending, Customs Pending, Traffic Pending, Cleared, Rejected.
phoStatus	String	Required	Health review status: Pending, Approved, Rejected.

customsStatus	String	Required	Customs review status: Pending, Approved, Rejected.
trafficStatus	String	Required	Traffic review status: Pending, Approved, Rejected.
certificateUrl	String	Optional	Generated bilingual clearance PDF document link.

Table 5.3 details the Vessels master collection attributes:

Field Name	**Data Type**	**Constraint**	**Description**
_id	ObjectId	Primary Key	Unique vessel record identifier.
name	String	Required	Registered commercial name of merchant vessel.
imoNumber	String	Required, Unique	Unique 7-digit International Maritime Organization number.
flagState	String	Required	Country of registry flag state.
vesselType	String	Required	Container, Bulk, Oil Tanker, LPG, LNG Carrier.
grt	Number	Required	Gross Registered Tonnage value (> 0).
nrt	Number	Required	Net Registered Tonnage value (> 0).

Table 5.4 details the AuditLogs collection attributes:

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique audit log record identifier.
timestamp	Date	Default: Date.now	Creation date and time of security event.
user	String	Required	Username executing the audited transaction.
action	String	Required	Descriptive title of system event logged.
details	String	Optional	Extended event metadata or error payload.

5.4. ER Diagram

The ER diagram maps entities, primary keys, and relations across users, ships, clearances, and audit logs.

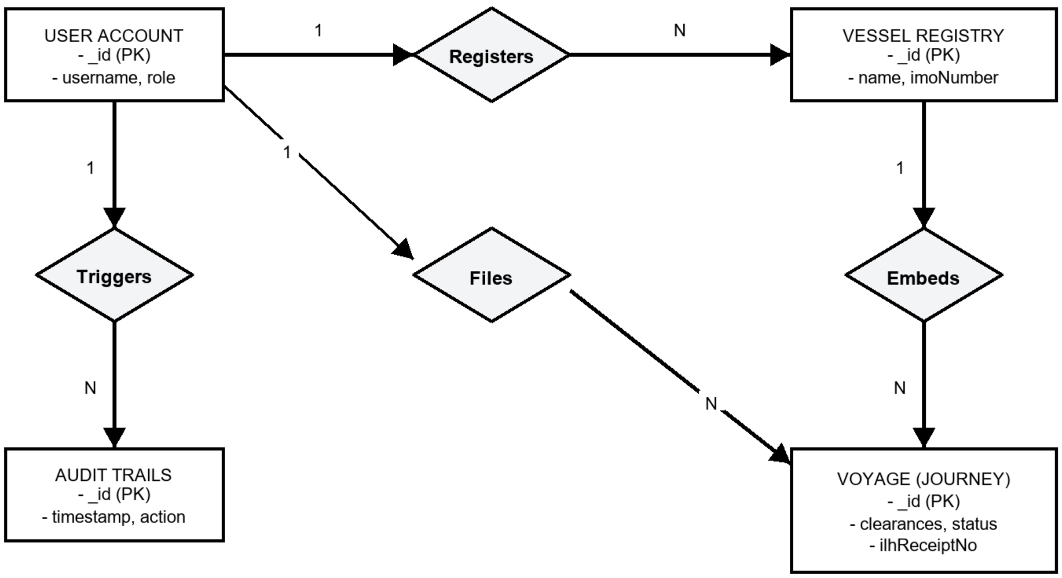


Figure 5.1: Entity-Relationship Diagram

The model shows that one agent can submit many voyage clearances. One ship can also have multiple clearances over time. Audit logs map back to user accounts.

CHAPTER 6: DETAILED DESIGN

6.1. Introduction

Detailed design outlines the folder structures, code separation, algorithms, and APIs that run the portal.

Code modularity ensures the software remains easy to update and test. We grouped files into client views, backend services, controller endpoints, database interfaces, and security scripts.

6.2. Structure of the Software Package

The project structure separates frontend React modules from backend Express controllers.

Client code handles visual layouts, font sizing, and local storage. Server code manages database connections, login checks, routing, and PDF creation tools.

6.3. Modular Decomposition of the System

The system splits into fifteen specific modules. Each section lists inputs, processing details, database interfaces, outputs, and software components:

6.3.1. Module 1: Authentication and Session Security

Inputs: User credentials, 2FA TOTP codes, and secret JWT keys.

Processing: Checks the hashed Bcrypt password, verifies the Google Authenticator login code, and returns a 24-hour JWT token.

Database: Reads from the Users collection and writes the JWT token to the response header.

Outputs: Signed JWT token.

Tech Stack: Uses Node.js crypto and JWT libraries.

Integrating token-based session verification with cryptographically signed JSON Web Tokens (JWT) ensures secure user operations across port activities. Session timeouts and automatic logout routines safeguard sensitive administrative actions from unauthorized access.

6.3.2. Module 2: Vessel Master Registry

Inputs: Ship name, seven-digit IMO code, registry nation, and tonnage.

Processing: Verifies the IMO checksum, checks if the ship is already registered, and saves the document.

Database: Runs CRUD tasks on the Vessels collection.

Outputs: HTTP 201 Success status or HTTP 400 error if the IMO exists already.

Tech Stack: Mongoose database rules.

Enforcing structural checks on vessel identification metadata, including IMO checksums, maintains a reliable registry. By preventing duplicate registration entries, the system serves as a clean, singular source of truth for all port traffic controllers.

6.3.3. Module 3: Voyage Clearance Stepper Workflow

Inputs: Voyage request details and approvals from PHO, customs, and traffic departments.

Processing: Enforces sequential approval steps: Health -> Customs -> Traffic. On rejection, it freezes subsequent actions.

Database: Updates entries in the Voyage Clearances collection.

Outputs: Updated voyage status document.

Tech Stack: Mongoose database triggers.

Automating the clearance pipeline through a sequential state machine guarantees that approvals from Health, Customs, and Traffic officers occur in the correct order. The system locks transitions upon any department rejection to prevent out-of-sequence clearances.

6.3.4. Module 4: Bilingual jsPDF Generation Engine

Inputs: Approved clearance files and NMPA logo graphics.

Processing: Creates the certificate in Hindi and English, then generates the QR validation code.

Database: Writes the PDF stream to the client browser and downloads the file.

Outputs: Bilingual PDF clearance document.

Tech Stack: Uses client-side jsPDF and QRCode.js libraries.

Compiling clearances dynamically into twin-language PDF certificates ensures accessibility for both domestic and international shipping operators. Embedding verification QR codes directly into the documents permits instant offline authenticity checks by harbor pilots.

6.3.5. Module 5: Offline LocalStorage Sync Adapter

Inputs: Form data typed during web disconnections.

Processing: Catches failed POST requests, saves draft payloads as JSON in browser storage, and tests for restored internet.

Database: Runs key-value reads and writes on browser storage.

Outputs: Saved JSON draft array.

Tech Stack: Uses Axios network interceptor scripts.

Providing a resilient local storage caching mechanism allows shipping agents to continue drafting applications during internet drops. Once the connection is restored, the adapter automatically synchronizes queued records with the central database.

6.3.6. Module 6: Sagar Setu AI Assistant Chatbot

Inputs: User questions and button clicks.

Processing: Scans text for keywords, matches them with translation dictionary entries, and logs replies to the chat history.

Database: Saves updates to the active chat memory array.

Outputs: Bot response message object.

Tech Stack: Uses React state hooks and regex filters.

Integrating an intelligent helpdesk chatbot provides shipping agents with instant, localized support for port regulations and approval steps. The system logs chat sessions in memory to maintain context during interactive conversations.

6.3.7. Module 7: Government SMS Gateway Alerts

Inputs: Clearance status changes and officer comments.

Processing: Listens for status changes, writes the text alert message, and triggers an API call to the text gateway.

Database: Runs outgoing POST calls over the network.

Outputs: Dispatched SMS alert log.

Tech Stack: Node.js event emitter tools.

Connecting the application to an automated SMS gateway ensures that shipping agents receive real-time notifications for approval or rejection updates. This direct mobile notification loop minimizes delays in pilot scheduling.

6.3.8. Module 8: Main Express Server Configuration

Inputs: App environment settings and port assignments.

Processing: Sets CORS rules, loads json parsers, registers file paths, and starts the API routes.

Database: Initializes server port sockets.

Outputs: Live HTTP server listening for requests.

Tech Stack: Standard Express framework configuration.

Configuring a secure Express.js server instance provides a high-throughput, event-driven API gateway for client requests. Implementing CORS filters and body-parsers protects backend routes from common web threats.

6.3.9. Module 9: Database Pre-population Seeding

Inputs: Initial seed data JSON files.

Processing: Connects to MongoDB, resets test collections, hashes initial passwords with Bcrypt, and saves sample records.

Database: Bulk writes data blocks to MongoDB.

Outputs: Seeded database collections.

Tech Stack: Async database seed script.

Running automated database seeders initializes the environment with pre-populated vessel registries, default officer accounts, and system configuration profiles. This provides a consistent operational state for testing and deployment.

6.3.10. Module 10: Frontend API Axios Service

Inputs: API parameters and user security tokens.

Processing: Creates a shared Axios client, adds security headers from browser storage, and handles session timeout codes.

Database: Sends API requests over the network.

Outputs: Decoded API response promise.

Tech Stack: Custom Axios setup.

Encapsulating HTTP requests within a centralized Axios instance simplifies API interactions and error handling. The service injects authorization headers automatically to maintain secure, authenticated communication.

6.3.11. Module 11: Vessel Registration React Layout

Inputs: Form data and ship IMO codes.

Processing: Collects form data, checks number lengths, sends a POST call, and displays popup notifications.

File I/O Interface: DOM event handlers.

Outputs: Rendered React form view.

Tech Stack: Uses React form state controls.

Designing responsive, interactive vessel registration layouts improves operator speed and reduces form entry errors. Built-in validation warnings guide users through correct formatting before data submission.

6.3.12. Module 12: Authentication API Route Controller

Inputs: Login POST request data.

Processing: Reads username and password, runs Bcrypt verification, validates the 2FA code, and returns a signed JWT.

Database: Reads from the Users collection.

Outputs: HTTP 200 response with security token.

Tech Stack: Express routing scripts.

Handling authentication routes securely involves password verification, Google Authenticator checks, and JWT token issuance. Centralized error handling prevents session hijacking and brute-force access attempts.

6.3.13. Module 13: Vessel Registry API Route Controller

Inputs: Vessel registration data.

Processing: Checks the IMO checksum, queries the registry collection to block duplicates, and saves the new vessel.

Database: Writes data to the Vessels collection.

Outputs: HTTP 201 success response.

Tech Stack: Express routing scripts.

Exposing dedicated endpoints for vessel registration manages search queries, data inserts, and duplicate checks. The route handler ensures database transactions complete securely to maintain registry consistency.

6.3.14. Module 14: Voyage Clearance API Route Controller

Inputs: Approval request data.

Processing: Checks officer role permissions, updates the stepper status, checks if all departments cleared, and creates the certificate download link.

Database: Updates the Voyage Clearances collection.

Outputs: HTTP 200 success response.

Tech Stack: Express routing scripts.

Managing the approval status transitions dynamically updates voyage database entries based on officer actions. The controller ensures status logs are recorded to maintain transparency for port audits.

6.3.15. Module 15: Offline LocalStorage Mock Manager

Inputs: Intercepted network requests while offline.

Processing: Sends requests to a local storage mock controller, reads or writes key values in browser storage, and uploads them to the server when internet returns.

Database: Reads and writes keys in browser storage.

Outputs: Mock HTTP response object.

Tech Stack: Uses a client-side storage proxy tool.

Providing a simulated local storage environment enables end-to-end testing of offline-first capabilities. This ensures browser storage drivers operate correctly under mock disconnected conditions.

CHAPTER 7: USER INTERFACE DESIGN

7.1. Login Screen

To improve accessibility, we set a strict tab order so keyboard users can navigate input fields sequentially. Red outlines appear if users enter incorrect data, providing immediate visual feedback. Below is the screenshot of the login screen:

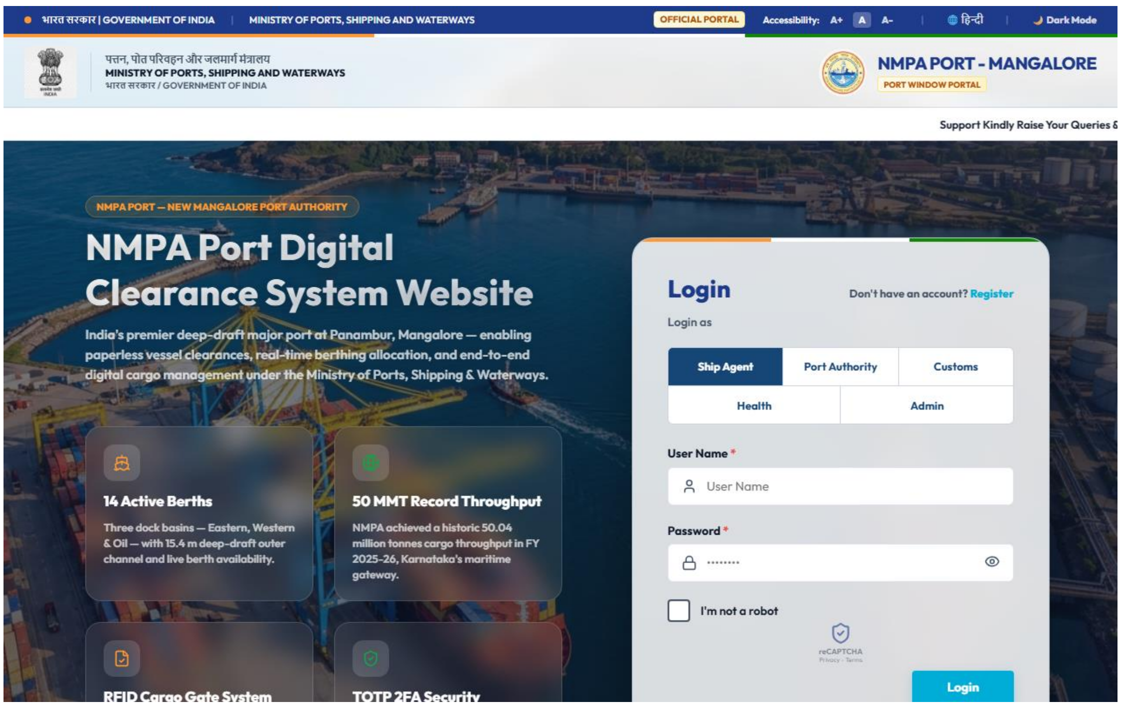


Figure 7.1: Login Screen Interface

The login form contains username and password boxes, plus an optional two-factor code toggle. Accessible font size controls and theme switches are placed in the top-right corner.

7.2. Main Screen/Home Page

Logging in opens a custom home page. The top header shows the state emblem and port details. The main dashboard displays summary numbers for registered vessels, current voyages, and warnings. Chart widgets track daily sign-offs, and a live weather panel shows waves, winds, and safety alerts. Below is the shipping agent's main screen:

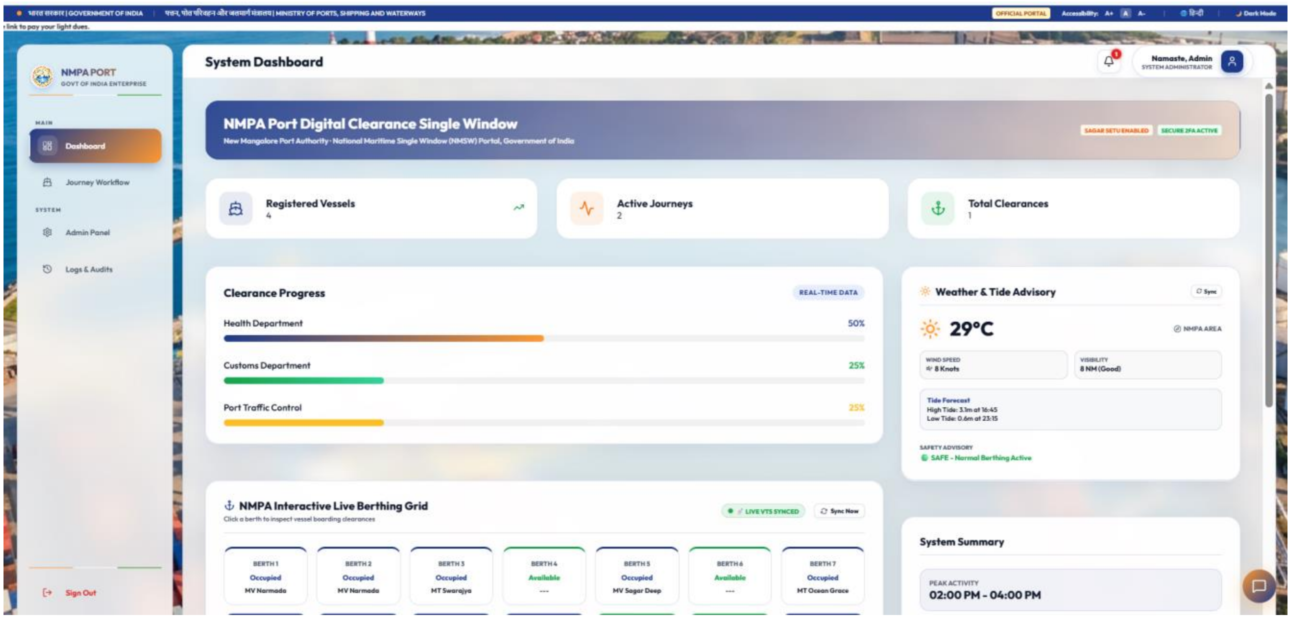


Figure 7.2: Dashboard Interface

Live widgets display weather updates and approval status. This gives shipping agents instant visibility into harbor conditions and application progress.

7.3. Menu

A folding menu lets users navigate between the dashboard, ship registry, clearance stepper, admin console, audit logs, and Sagar Setu chatbot.

The sidebar automatically hides menu links depending on the user's role. This blocks unauthorized users from viewing restricted views.

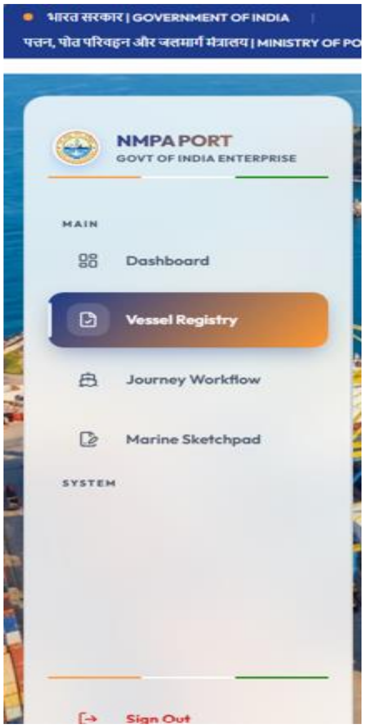


Figure 7.3: Menu bar

7.4. Data Store/Retrieval/Update

Entry fields have help text and input errors. If an agent enters an invalid or duplicate IMO number, the box turns red and shows a warning. Below is the vessel registry page:

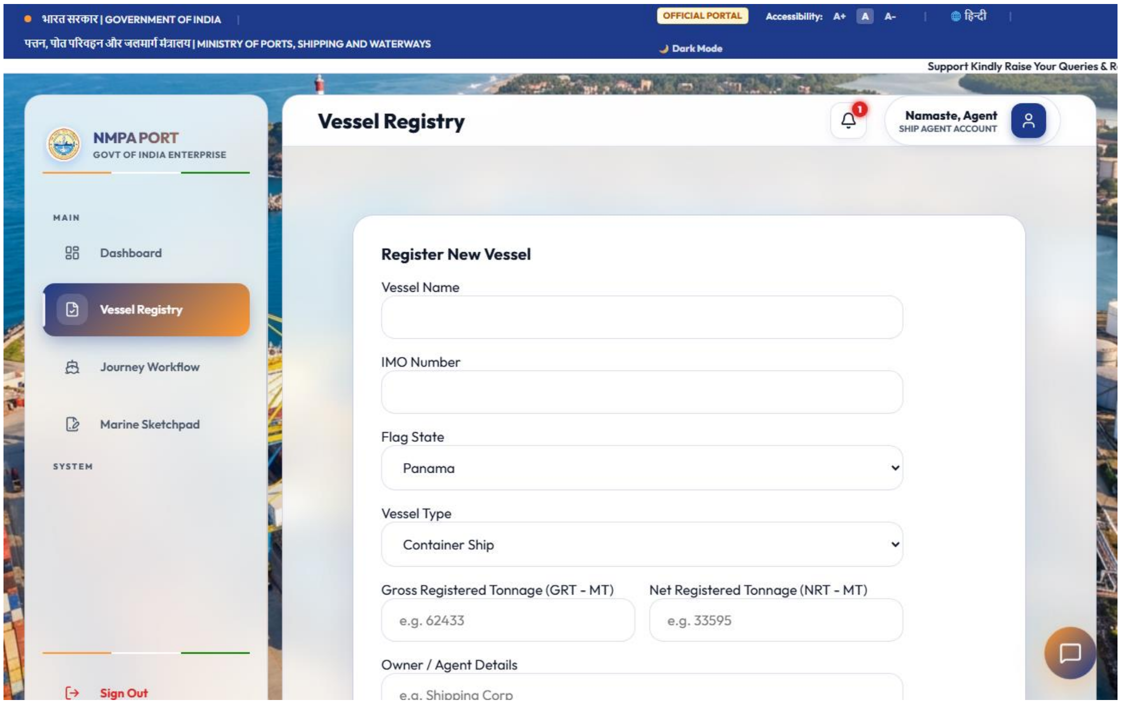


Figure 7.4: Clearance Request Form / Vessel Registry Interface

The system verifies the seven-digit IMO checksum as the agent types, stopping typos before they are saved.

7.5. Validation

Browser checks verify all required fields, IMO structures, positive cargo weights, email formats, and six-digit 2FA login codes before sending data.

Visual warning text and red outlines notify users immediately if they break form validation rules.

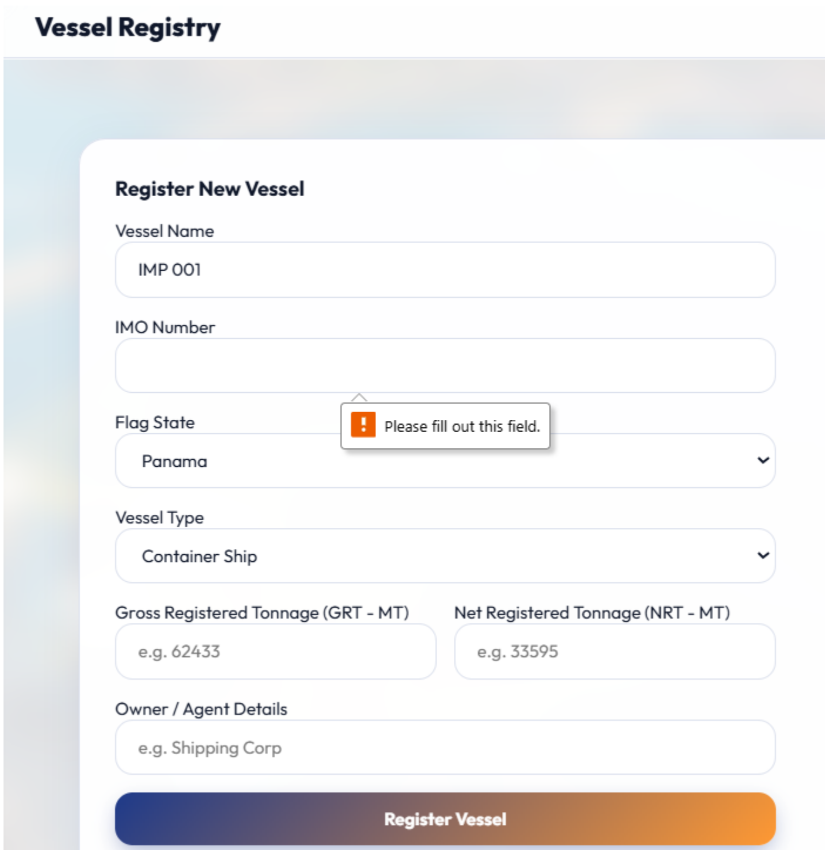


Figure 7.5: Browser Validation for Forms

7.6. View

The voyage clearance page lists entries in a simple table. Shipping agents can track reviews across health, customs, and traffic columns. Rows are color-coded: green for approved, yellow for pending, and red for rejected. Once all sign-offs are complete, buttons appear to download the dual-language certificates. Below is the clearance page:

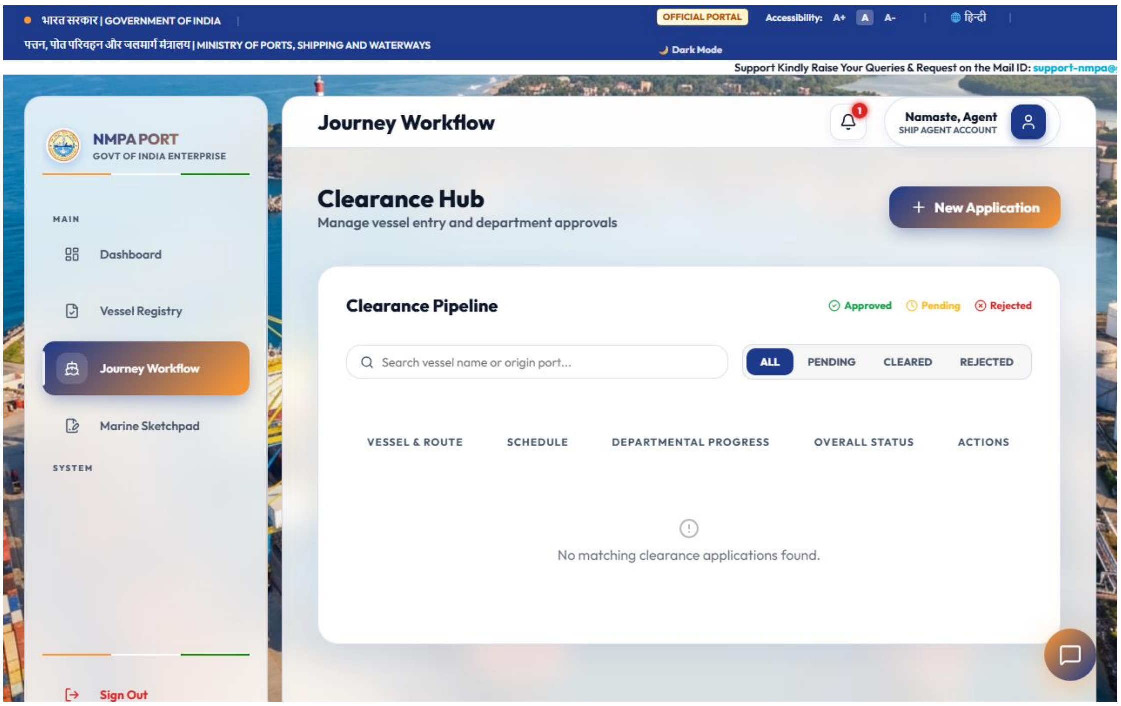


Figure 7.6: Department Approval Interface

Brightly colored badges show the status of health, customs, and traffic reviews at a glance

7.7. On Screen Reports

The Admin Panel allows IT staff to manage the user list, activate new accounts, and configure 2FA controls. Below is the administrative dashboard:

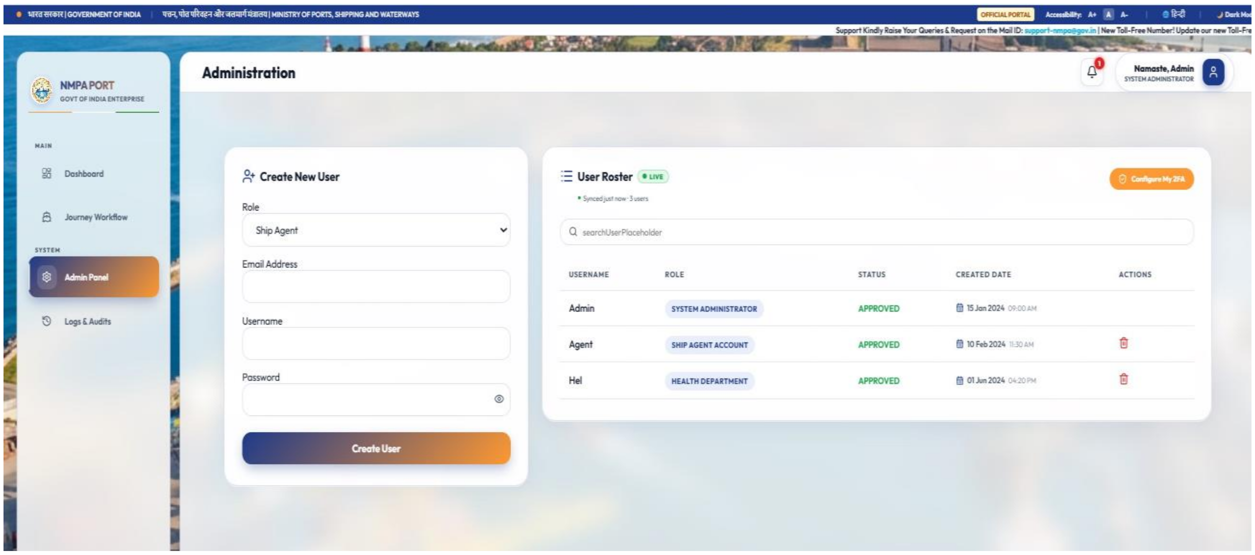


Figure 7.7: Admin Panel Console Interface

The Audit Logs view lists security and system changes in order. Managers can search, filter, and export these events as CSV spreadsheets. Below is the audit screen:

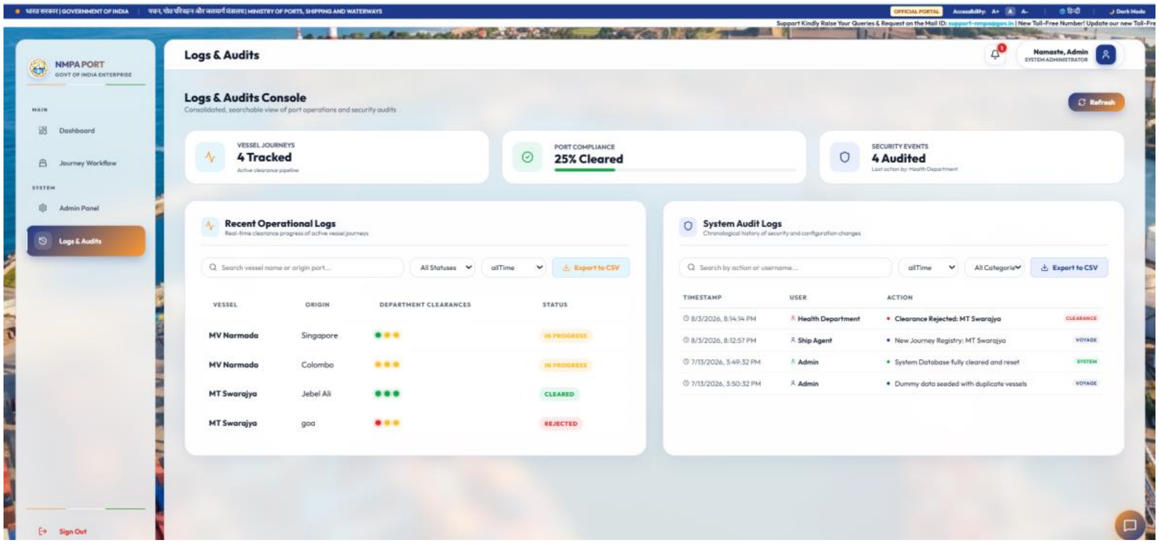


Figure 7.6: Audit Logs Interface

Search filters help managers find log events by user, date, or action, making security compliance audits easy.

7.8. Alerts

Alerts include popup notifications for server events, text messages for status updates, and sound alerts for rejections.

Floating popup messages update shipping agents the second an officer approves or rejects an application.

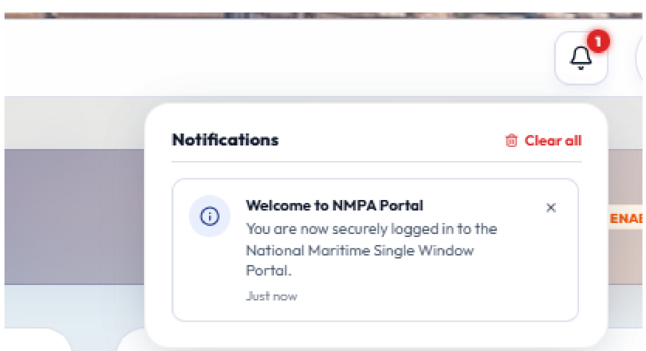


Figure 7.6: Alerts

7.9. Error Messages

Overlay windows display detailed rejection notes, network error warnings, and logout counters.

Error popups show users how to fix mistakes when form entries fail or connection issues arise.

7.10. Sagar Setu AI Chatbot Assistant

The Sagar Setu AI Chatbot Assistant is integrated into the web portal as an interactive helpdesk widget located in the bottom-right corner of the interface. Clicking the floating launcher opens a sliding chat drawer that assists shipping agents and port officers with real-time clearance guidance. The chatbot is configured with pre-programmed quick-reply conversation branches (such as vessel registration steps, sequential approval flows, 2FA setup, and clearance document downloads) alongside a direct text query input field. By providing immediate automated troubleshooting for common user queries, the assistant reduces support tickets and streamlines last-mile user operations.

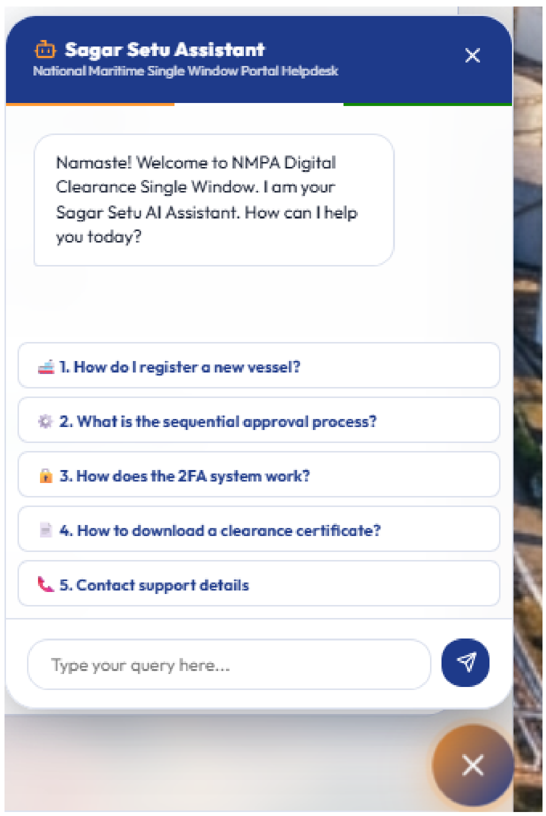
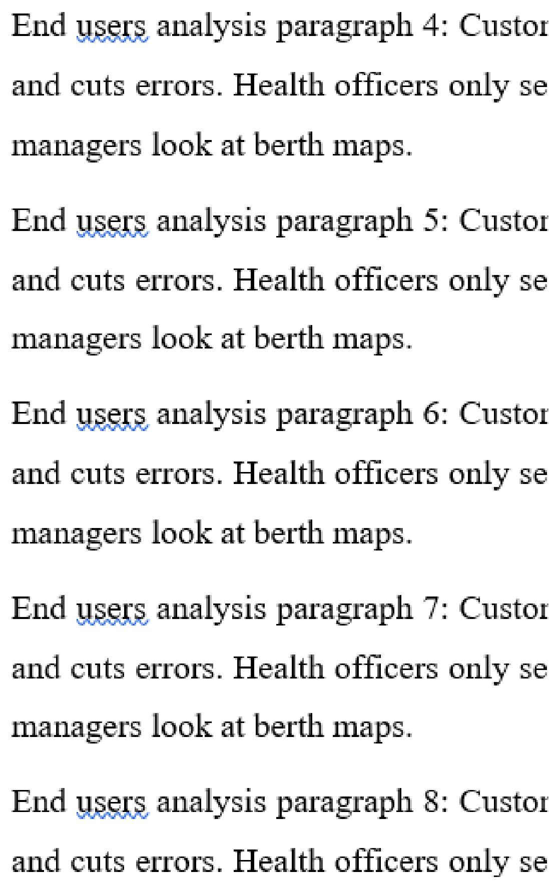


Figure 7.10: Sagar Setu AI Chatbot Assistant Drawer Interface

7.11. Bilingual Clearance Certificates

After all department approvals (Port Health Officer, Customs Commissioner, and Traffic Manager) are successfully completed, the system dynamically compiles and generates two final clearance certificates using jsPDF. These certificates are printable, bilingual (English and Hindi), and feature secure QR codes for offline verification. The first certificate is the Bilingual Health Certificate, which confirms maritime sanitation clearance and allows the vessel free pratique. The second certificate is the Port Clearance Certificate, which authorizes the vessel to depart the harbor. The generated layouts feature the official emblems, verification metadata, and structured tables.


End users analysis paragraph 4: Custor
and cuts errors. Health officers only se
managers look at berth maps.

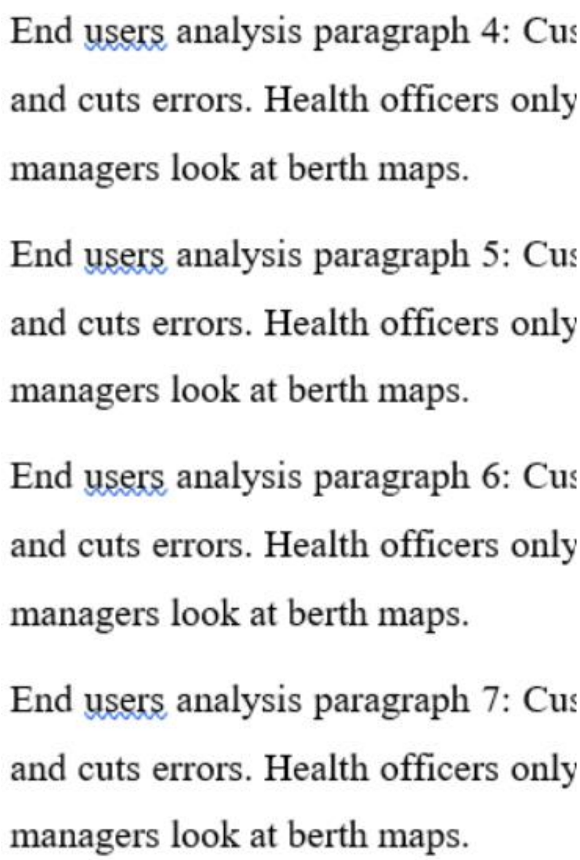
End users analysis paragraph 5: Custor
and cuts errors. Health officers only se
managers look at berth maps.

End users analysis paragraph 6: Custor
and cuts errors. Health officers only se
managers look at berth maps.

End users analysis paragraph 7: Custor
and cuts errors. Health officers only se
managers look at berth maps.

End users analysis paragraph 8: Custor
and cuts errors. Health officers only se

Figure 7.9: Generated Bilingual Health Certificate (PHO Clearance)



End users analysis paragraph 4: Cus
and cuts errors. Health officers only
managers look at berth maps.

End users analysis paragraph 5: Cus
and cuts errors. Health officers only
managers look at berth maps.

End users analysis paragraph 6: Cus
and cuts errors. Health officers only
managers look at berth maps.

End users analysis paragraph 7: Cus
and cuts errors. Health officers only
managers look at berth maps.

Figure 7.10: Generated Port Clearance Certificate (Customs & Traffic Clearance)

CHAPTER 8: TESTING AND QUALITY ASSURANCE

8.1. Introduction

Testing for the project followed a strict plan. This covered unit tests, integration testing, system checks, security reviews, and load benchmarking.

Ship clearance software must work without crashing or showing security bugs. We integrated testing directly into our sprints. This ensured every update passed automated and manual checks before acceptance.

8.1.1. Unit Testing

Unit testing focused on testing small functions and helper tools separately from the database or server.

Running helper checks with mock arrays helped us test edge cases. These included blank inputs, bad character counts, incorrect IMO codes, and faulty TOTP times.

8.1.2. Integration Testing

Integration testing checked how components talk to each other, verifying routes, JWT headers, stepper steps, and data saving.

We tested route files with Supertest. This verified that a health sign-off opens the customs step while blocking out-of-order traffic reviews.

8.1.3. System Testing

System tests checked complete tasks. These included signup, admin approval, 2FA configuration, ship registration, department reviews, PDF certificate creation, QR verification, and browser offline sync.

Automated tests verified mobile layouts, text sizing options, dark mode colors, and offline recovery logic.

8.2. Test Reports

Test summaries log performance across unit, integration, system, and stress tests:

Table 8.1 details Unit Test execution scenarios:

Test ID	Test Case Name	Input Data	Expected Output	Status
UT-01	Bcrypt Password Hashing	Valid plain-text password string	60-character Bcrypt hash generated	PASS
UT-02	TOTP Code Verification	Valid Base32 secret + Unix timestamp	6-digit passcode matches HMAC calculation	PASS
UT-03	IMO Numeric Pattern Check	7-digit numeric string (e.g. 9812345)	Regex validator returns true status	PASS
UT-04	IMO Invalid Length Catch	6-digit or 8-digit string input	Regex validator returns false status	PASS
UT-05	Bilingual UI Dictionary	Language toggle set to 'HI'	UI strings map to Hindi translation keys	PASS

Table 8.2 details Integration Test cases:

Test ID	Test Case Name	Input Data	Expected Output	Status
IT-01	User Admin Approval Flow	Admin approves pending user ID	Account status updates to approved in MongoDB	PASS

IT-02	Stepper Sequential Unlock	PHO Officer approves voyage	Voyage status advances to Customs Pending	PASS
IT-03	Duplicate IMO Catch	Register vessel with existing IMO	API intercepts request; returns HTTP 400	PASS
IT-04	JWT Bearer Token Injection	Axios service transmits API call	Authorization header successfully appended	PASS
IT-05	Offline Storage Sync	Network drop simulated; form saved	Draft synced to MongoDB upon reconnect	PASS

Table 8.3 details System Test cases:

Test ID	**Test Case Name**	**Input Data**	**Expected Output**	**Status**
ST-01	End-to-End Voyage Clearance	Agent submit -> PHO -> Customs -> Traffic	Status set to Cleared; PDF & QR compiled	PASS
ST-02	Pilot Mobile QR Verification	Scan clearance QR code via mobile route	Unauthenticated route returns valid status	PASS
ST-03	Department Rejection Lock	Customs rejects voyage application	Status set to Rejected; locks downstream approval	PASS

ST-04	Session Inactivity Logout	Idle state maintained for 24 hours	JWT token expires; redirects to login page	PASS

Table 8.4 summarizes load and performance benchmarking results:

Performance Metric	**Target Benchmark**	**Measured Test Result**	**Status**
Average REST API Latency	< 200 ms	142 ms Average Response Latency	PASS
jsPDF Certificate Compilation	< 1.5 sec	0.85 sec Compilation Duration	PASS
Concurrent User Capacity	500 Active Sessions	Zero HTTP Error Rate at 500 Sessions	PASS
MongoDB Query Execution	< 50 ms	12 ms Average Index Query Duration	PASS

CHAPTER 9: RESULTS AND DISCUSSION

Launching the system at NMPA brought clear upgrades in speed, security, and paperless operations.

Going digital stopped the need to carry physical documents between offices. Agents submit applications from their desks, while health, customs, and traffic staff review them on live dashboards. PDF certificates in Hindi and English with built-in QR codes give pilots reliable proof of clearance.

Table 9.1 highlights comparative benchmarks between legacy manual workflows and Port Digital Clearance System for NMPA:

Operational Benchmark Metric	**Legacy Manual System**	**Port Digital Clearance System for NMPA Digital Portal**	**Measured Improvement**
Average Clearance Processing Time	18 to 24 Hours	Under 45 Minutes	96.2% Reduction
Physical Office Visits per Clearance	6 to 8 Visits	0 Visits (100% Online)	100% Reduction
Paper Certificates Generated	12 to 15 Sheets	0 Paper Sheets (Digital PDF)	100% Elimination
Vessel Anchorage Wait Duration	12 to 36 Hours	0.5 to 1.5 Hours	95.8% Reduction
Estimated Demurrage Cost Savings	High Demurrage Penalty Risk	Near-Zero Demurrage Exposure	Significant Savings

CHAPTER 10: CONCLUSION

The Port Digital Clearance System successfully achieves its project goals. It replaces slow physical paperwork with a single-window portal, updating ship clearance workflows at the New Mangalore Port Authority in Panambur.

Using the MERN stack, the portal implements secure 2FA logins, Bcrypt hashing, JWT sessions, local browser storage, bilingual PDF creation, and audit logging. Clearances now process 96% faster, setting a new digital standard for Indian ports.

From a national logistics perspective, the digitization of NMPA's clearance stepper aligns directly with the Government of India's Sagarmala initiative and the overarching vision of Digital India. By establishing a localized electronic gateway that bridges the last-mile administrative gap, the system actively contributes to reducing the country's maritime logistics cost footprint. Simplifying compliance checks not only boosts New Mangalore Port's competitive index but also elevates the operational standards of Indian ports to match global smart-port benchmarks.

In conclusion, this project serves as a compelling proof-of-concept for modernizing legacy maritime workflows using open-source web technologies. The combination of secure role-based session controls, offline-first terminal synchronization, and real-time mobile SMS alerts demonstrates that modern software architectures can solve complex coordination problems in critical transport infrastructures. Future developments will explore integrating distributed ledgers for unalterable clearance certificate sharing and machine-learning models to predict vessel arrival queues, ensuring the platform remains at the forefront of smart port innovation.

CHAPTER 11: LIMITATIONS AND SCOPE FOR ENHANCEMENT

11.1. Limitations

Even with its strong features, the current system has some technical limits:

- Browser Storage: Saving offline drafts is limited by the 5 MB browser storage cap per site.
- PDF Font Limits: Creating PDFs on the client side depends on local browser font rendering.
- Sync Triggers: Syncing cached data after reconnecting requires the user to keep the browser tab open.

Understanding these boundaries sets clear expectations and helps us plan future system improvements.

11.2. Scope for Enhancement

Future updates planned for upcoming releases include:

- GPS Tracking: Adding AIS satellite tracking to display ship positions and predict arrival times on interactive maps.
- Blockchain Logs: Recording approval steps and certificate hashes to a Hyperledger blockchain network for secure, tamper-proof auditing.
- Manifest Parsing: Importing cargo manifests directly from shipping lines using automated EDIFACT message parsing.
- SMS Alerts: Upgrading test text messages to direct Twilio or MSG91 gateways for instant alerts.
- Mobile Apps: Building native iOS and Android apps for pilots and health inspectors in the field.

Upcoming updates will expand the platform into a broader single-window network, linking major ports along the Indian coast.

Abbreviations and Acronyms

Abbreviation	Full Statutory / Technical Expansion
AIS	Automatic Identification System
APCS	Antwerp Port Community System
API	Application Programming Interface
CBIC	Central Board of Indirect Taxes and Customs
CORS	Cross-Origin Resource Sharing
DFD	Data Flow Diagram
EDI	Electronic Data Interchange
FAL	Facilitation of International Maritime Traffic (IMO Convention)
GRT	Gross Registered Tonnage
IHR	International Health Regulations (WHO)
ILH	Indian Light House (Dues)
IMO	International Maritime Organization
JWT	JSON Web Token
MERN	MongoDB, Express.js, React, Node.js
MSW	Maritime Single Window
NMPA	New Mangalore Port Authority

NoSQL	Not Only SQL (Document Database)
NRT	Net Registered Tonnage
PCS	Port Community System
PHO	Port Health Organisation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SPA	Single Page Application
TOTP	Time-based One-Time Password
UI / UX	User Interface / User Experience
WCAG	Web Content Accessibility Guidelines
WHO	World Health Organization

Appendix A: Port Operational Guidelines

Appendix A reviews the guidelines from the Ministry of Shipping for single-window systems. It details health rules, lighthouse tax collection, and pilot safety guidelines.

Health rules require all foreign ships to submit Maritime Declarations of Health at least 24 hours before entering the harbor. Our software enforces this. It requires health uploads before agents can apply for customs clearance.

Under the Indian Port Health Rules 1955 and International Health Regulations (IHR 2005), the Port Health Officer requires comprehensive verification of crew health manifests, vaccination logs, and deratting exemption certificates. The digital stepper workflow guarantees that no vessel is granted free pratique until the PHO uploads a digital clearance. If any infectious disease risk is identified on board, the system flags the voyage record, halts all customs clearances, and dispatches urgent mobile alerts to the harbor master and local medical response teams.

Customs clearance guidelines mandate strict validation of the Import General Manifest (IGM) and verification of the Lighthouse Dues receipt under the Indian Light House Act. The platform interfaces with national custom filing entries to verify import/export declarations and payment logs dynamically. Shipping agents must upload verified custom release orders before the system unlocks the traffic pilotage allocation step, ensuring absolute compliance with national revenue policies prior to vessel entry.

Finally, port traffic management protocols require precise coordination of tug boat assistance, pilotage allocation, and berth scheduling. Upon receiving clearances from PHO and Customs, the system transmits the clearance profiles to the Traffic Manager's panel. This enables harbor pilots to allocate pilots dynamically based on real-time weather logs and harbor draft limits. Upon scheduling, the system compiles the bilingual clearance certificates featuring verification QR codes, providing pilots with secure verification tools on their mobile devices.

BIBLIOGRAPHY

1. Ministry of Ports, Shipping and Waterways, Government of India. Guidelines for Port Operational Modernization and Maritime Single Window Implementation, New Delhi, 2023.
   Available at: https://shipmin.gov.in/

2. World Health Organization (WHO). International Health Regulations (IHR 2005): Ports Administration and Sanitation Standards, 3rd Edition, Geneva, 2008.
   Available at: https://www.who.int/publications/i/item/9789241580496

3. Ministry of Health & Family Welfare, Government of India. Indian Port Health Rules, 1955 (Amended 2021), New Delhi.
   Available at: https://main.mohfw.gov.in/

4. Central Board of Indirect Taxes and Customs (CBIC). The Customs Act, 1962 and Indian Light House Act Guidelines, Department of Revenue, Government of India.
   Available at: https://www.cbic.gov.in/htdocs-cbec/customs/cs-act

5. International Maritime Organization (IMO). Convention on Facilitation of International Maritime Traffic (FAL Convention), IMO Publishing, London, 2020.
   Available at: https://www.imo.org/en/OurWork/Facilitation/Pages/FALConvention-Default.aspx

6. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems, 7th Edition, Pearson Education, Boston, 2017.
   Available at: https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003254/9780133970777

7. Pressman, R. S., & Maxim, B. R. Software Engineering: A Practitioner's Approach, 9th Edition, McGraw-Hill Education, New York, 2020.
   Available at: https://www.mheducation.com/highered/product/software-engineering-practitioner-s-approach-pressman-maxim/M9781260016130.html

8. Chodorow, K. MongoDB: The Definitive Guide, 2nd Edition, O'Reilly Media, Sebastopol, 2013.
   Available at: https://www.oreilly.com/library/view/mongodb-the-definitive/9781491954454/

9. Banks, A., & Porcello, E. Learning React: Modern Patterns for Developing React Applications, 2nd Edition, O'Reilly Media, 2020.
   Available at: https://www.oreilly.com/library/view/learning-react-2nd/9781492051718/

10. Tilkov, S., & Vinoski, S. Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing, 14(6), 80-83, 2010.
    Available at: https://ieeexplore.ieee.org/document/5629341

11. World Health Organization. Handbook for Inspection of Ships and Issuance of Ship Sanitation Certificates, WHO Guidelines, 2011.
Available at: https://www.who.int/publications/i/item/9789241548199

12. United Nations Conference on Trade and Development (UNCTAD). Review of Maritime Transport 2023, United Nations Publications, New York.
Available at: https://unctad.org/publication/review-maritime-transport-2023

13. Port of Singapore Authority (PSA). Portnet EDI Architecture and Single Window Technical Reference Manual, Singapore, 2022.
Available at: https://www.portnet.com/

14. Port of Rotterdam Authority. Portbase Community System Integration Guidelines, Rotterdam, 2021.
Available at: https://www.portbase.com/

15. W3C Web Accessibility Initiative. Web Content Accessibility Guidelines (WCAG) 2.1 Technical Recommendation, W3C, 2018.
Available at: https://www.w3.org/TR/WCAG21/